Assignment 7:

```
/**
 *
================================================================================
 * KBTCOE Connect — Frontend Form Validation Module (validation.js)
 * Assignment 07: Frontend Validation
 *
 * Demonstrates:
 * 1. Event Listeners: 'submit', 'input', 'blur', 'change', 'reset'
 * 2. Event Handling: event.preventDefault() to intercept native submission
 * 3. Modular Custom Validation Functions with descriptive return objects
 * 4. Regular Expressions for format checks (Email, Student ID, Full Name)
 * 5. Password Complexity & Cross-Field Matching (Password === Confirm Password)
 * 6. Accessible DOM Manipulation: aria-invalid, aria-describedby, textContent
 * 7. Real-Time Feedback: Instant error clearing upon user correction
 * 8. Safe Data Presentation: No innerHTML injection, no logging of sensitive
fields
 * 9. Universal Module Export for Node.js automated unit testing
 *
================================================================================
 */

"use strict";

//
================================================================================
// 1. CONFIGURABLE VALIDATION RULES (Configurable demonstration formats)
//
================================================================================
/**
 * Institutional validation rules are kept configurable here rather than
 * hardcoded to strict assumptions. In a real deployment, these values could
 * be injected via environment configurations or college portal settings.
 */
const VALIDATION_CONFIG = {
  // Demonstration Student ID pattern:
  // Format: KBT-YYYY-DEPTXX (e.g. KBT-2023-CS01, KBT-2024-IT12, KBT-2025-ME03)
  studentId: {
    pattern: /^KBT-\d{4}-[A-Z0-9]{2,4}$/i,
    description: "Format: 'KBT-YYYY-DEPTXX' (e.g., KBT-2023-CS01)"
  },
  // Full Name: 2 to 60 characters, alphabetic letters, spaces, hyphens, periods
  fullName: {
    minLength: 2,
    maxLength: 60,
    pattern: /^[a-zA-Z\s.'-]+$/
  },
  // Email: Standard sensible email format without being overly restrictive
  email: {
    pattern: /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/
  },
```

```
  // Password: At least 8 characters, containing at least one letter and one
number
  password: {
    minLength: 8,
    maxLength: 128,
    pattern: /^(?=.*[A-Za-z])(?=.*\d)/
  }
};

//
================================================================================
// 2. PURE VALIDATION FUNCTIONS (Easily unit-testable)
//
================================================================================

/**
 * Validates the Full Name field.
 * - Trims leading and trailing whitespace.
 * - Checks for non-empty value.
 * - Enforces minimum and maximum length limits.
 * - Validates permitted name characters (letters, spaces, hyphens).
 *
 * @param {string} name - Raw user input
 * @returns {{ isValid: boolean, message: string, sanitizedValue: string }}
 */
function validateFullName(name) {
  const sanitized = typeof name === "string" ? name.trim() : "";

  if (!sanitized) {
    return {
      isValid: false,
      message: "Full name is required.",
      sanitizedValue: ""
    };
  }

  if (sanitized.length < VALIDATION_CONFIG.fullName.minLength) {
    return {
      isValid: false,
      message: `Full name must be at least
${VALIDATION_CONFIG.fullName.minLength} characters long.`,
      sanitizedValue: sanitized
    };
  }

  if (sanitized.length > VALIDATION_CONFIG.fullName.maxLength) {
    return {
      isValid: false,
      message: `Full name cannot exceed ${VALIDATION_CONFIG.fullName.maxLength}
characters.`,
      sanitizedValue: sanitized
    };
  }
```

```
  if (!VALIDATION_CONFIG.fullName.pattern.test(sanitized)) {
    return {
      isValid: false,
      message: "Full name can only contain alphabetic letters, spaces, hyphens,
and periods.",
      sanitizedValue: sanitized
    };
  }

  return {
    isValid: true,
    message: "",
    sanitizedValue: sanitized
  };
}

/**
 * Validates the Student ID field against configurable demonstration format.
 * - Trims whitespace.
 * - Checks non-empty.
 * - Validates against regex pattern.
 *
 * @param {string} id - Raw user input
 * @returns {{ isValid: boolean, message: string, sanitizedValue: string }}
 */
function validateStudentId(id) {
  const sanitized = typeof id === "string" ? id.trim() : "";

  if (!sanitized) {
    return {
      isValid: false,
      message: "Student ID is required.",
      sanitizedValue: ""
    };
  }

  if (!VALIDATION_CONFIG.studentId.pattern.test(sanitized)) {
    return {
      isValid: false,
      message: `Invalid Student ID format.
${VALIDATION_CONFIG.studentId.description}.`,
      sanitizedValue: sanitized
    };
  }

  return {
    isValid: true,
    message: "",
    sanitizedValue: sanitized.toUpperCase()
  };
}

/**
 * Validates the Email Address field.
```

```
 * - Trims whitespace.
 * - Checks non-empty.
 * - Verifies reasonable structure (user@domain.tld).
 * Note: Clearly documents that format check does NOT verify mailbox existence.
 *
 * @param {string} email - Raw user input
 * @returns {{ isValid: boolean, message: string, sanitizedValue: string }}
 */
function validateEmail(email) {
  const sanitized = typeof email === "string" ? email.trim() : "";

  if (!sanitized) {
    return {
      isValid: false,
      message: "Email address is required.",
      sanitizedValue: ""
    };
  }

  if (!VALIDATION_CONFIG.email.pattern.test(sanitized)) {
    return {
      isValid: false,
      message: "Please enter a valid email address (e.g.,
student@example.com).",
      sanitizedValue: sanitized
    };
  }

  return {
    isValid: true,
    message: "",
    sanitizedValue: sanitized.toLowerCase()
  };
}

/**
 * Validates the Branch / Department selection.
 * Ensures a valid option is selected and not the empty placeholder.
 *
 * @param {string} branch - Selected value
 * @returns {{ isValid: boolean, message: string }}
 */
function validateBranch(branch) {
  const sanitized = typeof branch === "string" ? branch.trim() : "";

  if (!sanitized || sanitized === "") {
    return {
      isValid: false,
      message: "Please select your engineering department/branch."
    };
  }

  const validBranches = [
    "Computer Engineering",
```

```
    "Information Technology",
    "Artificial Intelligence & Data Science",
    "Electronics & Telecommunication",
    "Mechanical Engineering",
    "Civil Engineering",
    "Instrumentation Engineering"
  ];

  if (!validBranches.includes(sanitized)) {
    return {
      isValid: false,
      message: "Please select a recognized engineering department from the
list."
    };
  }

  return {
    isValid: true,
    message: ""
  };
}

/**
 * Validates the Year of Study selection.
 *
 * @param {string} year - Selected value
 * @returns {{ isValid: boolean, message: string }}
 */
function validateYear(year) {
  const sanitized = typeof year === "string" ? year.trim() : "";

  if (!sanitized || sanitized === "") {
    return {
      isValid: false,
      message: "Please select your current year of study."
    };
  }

  const validYears = [
    "First Year (FE)",
    "Second Year (SE)",
    "Third Year (TE)",
    "Final Year (BE)"
  ];

  if (!validYears.includes(sanitized)) {
    return {
      isValid: false,
      message: "Please select a valid academic year from the options."
    };
  }

  return {
    isValid: true,
```

```
    message: ""
  };
}

/**
 * Validates Password strength and length.
 * Does not trim passwords because leading/trailing spaces can be intentional.
 *
 * @param {string} password - Raw password
 * @returns {{ isValid: boolean, message: string }}
 */
function validatePassword(password) {
  const pwd = typeof password === "string" ? password : "";

  if (!pwd) {
    return {
      isValid: false,
      message: "Password is required."
    };
  }

  if (pwd.length < VALIDATION_CONFIG.password.minLength) {
    return {
      isValid: false,
      message: `Password must be at least
${VALIDATION_CONFIG.password.minLength} characters long.`
    };
  }

  if (pwd.length > VALIDATION_CONFIG.password.maxLength) {
    return {
      isValid: false,
      message: `Password cannot exceed ${VALIDATION_CONFIG.password.maxLength}
characters.`
    };
  }

  if (!VALIDATION_CONFIG.password.pattern.test(pwd)) {
    return {
      isValid: false,
      message: "Password must contain at least one letter and at least one
number."
    };
  }

  return {
    isValid: true,
    message: ""
  };
}

/**
 * Validates that Confirm Password matches Password.
 *
```

```
 * @param {string} password - Primary password
 * @param {string} confirmPassword - Confirm password input
 * @returns {{ isValid: boolean, message: string }}
 */
function validateConfirmPassword(password, confirmPassword) {
  const pwd = typeof password === "string" ? password : "";
  const cpwd = typeof confirmPassword === "string" ? confirmPassword : "";

  if (!cpwd) {
    return {
      isValid: false,
      message: "Please confirm your password."
    };
  }

  if (pwd !== cpwd) {
    return {
      isValid: false,
      message: "Passwords do not match. Please re-enter identical passwords."
    };
  }

  return {
    isValid: true,
    message: ""
  };
}

/**
 * Validates that the Terms and Conditions checkbox is checked.
 *
 * @param {boolean} isChecked - Checkbox checked state
 * @returns {{ isValid: boolean, message: string }}
 */
function validateTerms(isChecked) {
  if (!isChecked) {
    return {
      isValid: false,
      message: "You must accept the terms, code of conduct, and academic
integrity policies to proceed."
    };
  }

  return {
    isValid: true,
    message: ""
  };
}

//
================================================================================
// 3. ACCESSIBLE UI ERROR & SUCCESS RENDERING (Browser DOM methods)
//
================================================================================
```

```
/**
 * Sets or clears error state on a specific form input field.
 * Uses textContent and ARIA attributes for accessibility.
 *
 * @param {HTMLElement} inputElement - Input, select, or checkbox element
 * @param {HTMLElement} errorElement - Dedicated error message span
 * @param {boolean} isValid - Validity status
 * @param {string} errorMessage - Error text to display if invalid
 */
function setFieldValidationUI(inputElement, errorElement, isValid, errorMessage
= "") {
  if (!inputElement || !errorElement) return;

  const formGroup = inputElement.closest(".form-group");

  if (isValid) {
    // Accessible attributes
    inputElement.setAttribute("aria-invalid", "false");
    inputElement.classList.remove("input-error");
    inputElement.classList.add("input-success");

    if (formGroup) {
      formGroup.classList.remove("has-error");
      formGroup.classList.add("has-success");
    }

    // Clear error message cleanly
    errorElement.textContent = "";
    errorElement.classList.remove("active");
  } else {
    // Accessible attributes
    inputElement.setAttribute("aria-invalid", "true");
    inputElement.classList.remove("input-success");
    inputElement.classList.add("input-error");

    if (formGroup) {
      formGroup.classList.remove("has-success");
      formGroup.classList.add("has-error");
    }

    // Assign text safely using textContent (prevents XSS)
    errorElement.textContent = errorMessage;
    errorElement.classList.add("active");
  }
}

/**
 * Clears validation styling and messages from a field.
 *
 * @param {HTMLElement} inputElement
 * @param {HTMLElement} errorElement
 */
function clearFieldValidationUI(inputElement, errorElement) {
```

```
  if (!inputElement || !errorElement) return;

  const formGroup = inputElement.closest(".form-group");

  inputElement.removeAttribute("aria-invalid");
  inputElement.classList.remove("input-error", "input-success");

  if (formGroup) {
    formGroup.classList.remove("has-error", "has-success");
  }

  errorElement.textContent = "";
  errorElement.classList.remove("active");
}

//
================================================================================
// 4. FORM CONTROLLER & EVENT LISTENERS
//
================================================================================

/**
 * Encapsulates the entire registration form controller logic.
 */
function initRegistrationForm() {
  const form = document.getElementById("registrationForm");
  if (!form) return;

  // Form DOM Field References
  const fields = {
    fullName: {
      input: document.getElementById("fullName"),
      error: document.getElementById("fullNameError"),
      validate: () => validateFullName(fields.fullName.input.value)
    },
    studentId: {
      input: document.getElementById("studentId"),
      error: document.getElementById("studentIdError"),
      validate: () => validateStudentId(fields.studentId.input.value)
    },
    email: {
      input: document.getElementById("email"),
      error: document.getElementById("emailError"),
      validate: () => validateEmail(fields.email.input.value)
    },
    branch: {
      input: document.getElementById("branch"),
      error: document.getElementById("branchError"),
      validate: () => validateBranch(fields.branch.input.value)
    },
    year: {
      input: document.getElementById("year"),
      error: document.getElementById("yearError"),
      validate: () => validateYear(fields.year.input.value)
```

```
    },
    password: {
      input: document.getElementById("password"),
      error: document.getElementById("passwordError"),
      validate: () => validatePassword(fields.password.input.value)
    },
    confirmPassword: {
      input: document.getElementById("confirmPassword"),
      error: document.getElementById("confirmPasswordError"),
      validate: () => validateConfirmPassword(
        fields.password.input.value,
        fields.confirmPassword.input.value
      )
    },
    terms: {
      input: document.getElementById("terms"),
      error: document.getElementById("termsError"),
      validate: () => validateTerms(fields.terms.input.checked)
    }
  };

  const successCard = document.getElementById("successFeedback");
  const summaryBox = document.getElementById("submittedDataSummary");
  const formCard = document.getElementById("formCard");
  const resetFormBtn = document.getElementById("resetFormBtn");
  const registerAnotherBtn = document.getElementById("registerAnotherBtn");

  /**
   * Validates a single field and updates its visual UI.
   * @param {string} fieldKey
   * @returns {boolean} Whether the field is valid
   */
  function validateField(fieldKey) {
    const field = fields[fieldKey];
    if (!field || !field.input) return true;

    const result = field.validate();
    setFieldValidationUI(field.input, field.error, result.isValid,
result.message);
    return result.isValid;
  }

  // --- Attach Real-Time Listeners for Responsive Feedback ---

  // Text inputs: validate on blur, and clear errors on input if correcting
  ["fullName", "studentId", "email"].forEach(key => {
    const field = fields[key];
    if (!field.input) return;

    field.input.addEventListener("blur", () => {
      // Validate on blur when user leaves the field
      validateField(key);
    });
```

```
    field.input.addEventListener("input", () => {
      // If the field currently has an error, re-validate immediately to clear
it as soon as user fixes it
      if (field.input.classList.contains("input-error")) {
        validateField(key);
      }
    });
  });

  // Dropdowns: validate on change
  ["branch", "year"].forEach(key => {
    const field = fields[key];
    if (!field.input) return;

    field.input.addEventListener("change", () => {
      validateField(key);
    });
  });

  // Password fields: dynamic interaction
  if (fields.password.input) {
    fields.password.input.addEventListener("blur", () => {
      validateField("password");
      // If confirm password already has a value, re-check match
      if (fields.confirmPassword.input.value) {
        validateField("confirmPassword");
      }
    });

    fields.password.input.addEventListener("input", () => {
      if (fields.password.input.classList.contains("input-error")) {
        validateField("password");
      }
      if (fields.confirmPassword.input.value) {
        validateField("confirmPassword");
      }
    });
  }

  if (fields.confirmPassword.input) {
    fields.confirmPassword.input.addEventListener("blur", () => {
      validateField("confirmPassword");
    });

    fields.confirmPassword.input.addEventListener("input", () => {
      if (fields.confirmPassword.input.classList.contains("input-error")) {
        validateField("confirmPassword");
      }
    });
  }

  // Terms checkbox: validate on change
  if (fields.terms.input) {
    fields.terms.input.addEventListener("change", () => {
```

```
    validateField("terms");
  });
}

// --- Form Reset Handler ---
function resetAllValidation() {
  Object.keys(fields).forEach(key => {
    const field = fields[key];
    clearFieldValidationUI(field.input, field.error);
  });

  if (successCard) {
    successCard.classList.remove("active");
    successCard.setAttribute("hidden", "true");
  }

  if (formCard) {
    formCard.style.display = "block";
  }

  form.reset();
  fields.fullName.input.focus();
}

if (resetFormBtn) {
  resetFormBtn.addEventListener("click", resetAllValidation);
}

if (registerAnotherBtn) {
  registerAnotherBtn.addEventListener("click", () => {
    resetAllValidation();
    window.scrollTo({ top: 0, behavior: "smooth" });
  });
}

// --- Form Submission Event Handler ---
form.addEventListener("submit", event => {
  // Prevent standard HTTP form submission
  event.preventDefault();

  let isFormValid = true;
  let firstInvalidField = null;

  // Validate all fields in logical order
  const fieldOrder = [
    "fullName",
    "studentId",
    "email",
    "branch",
    "year",
    "password",
    "confirmPassword",
    "terms"
  ];
```

```
    fieldOrder.forEach(key => {
      const valid = validateField(key);
      if (!valid) {
        isFormValid = false;
        if (!firstInvalidField) {
          firstInvalidField = fields[key].input;
        }
      }
    });

    // If validation fails, shift focus to the first invalid field for
accessibility
    if (!isFormValid) {
      if (firstInvalidField) {
        firstInvalidField.focus();
        // Announce error to screen readers
        const alertLiveRegion = document.getElementById("formLiveAlert");
        if (alertLiveRegion) {
          alertLiveRegion.textContent = "Please correct the highlighted errors
before submitting the registration form.";
        }
      }
      return;
    }

    // --- All Validations Passed ---
    // Prepare sanitized summary for demonstration display
    const sanitizedName =
validateFullName(fields.fullName.input.value).sanitizedValue;
    const sanitizedId =
validateStudentId(fields.studentId.input.value).sanitizedValue;
    const sanitizedEmail =
validateEmail(fields.email.input.value).sanitizedValue;
    const selectedBranch = fields.branch.input.value;
    const selectedYear = fields.year.input.value;

    // Populate the demonstration summary safely using textContent
    if (summaryBox) {
      summaryBox.replaceChildren();

      const items = [
        { label: "Full Name", value: sanitizedName },
        { label: "Student ID (Demonstration)", value: sanitizedId },
        { label: "Email Address", value: sanitizedEmail },
        { label: "Department / Branch", value: selectedBranch },
        { label: "Year of Study", value: selectedYear },
        { label: "Account Status", value: "Verified Client-Side (Demo Record)" }
      ];

      items.forEach(item => {
        const row = document.createElement("div");
        row.className = "summary-row";
```

```
        const labelSpan = document.createElement("strong");
        labelSpan.className = "summary-label";
        labelSpan.textContent = `${item.label}: `;

        const valueSpan = document.createElement("span");
        valueSpan.className = "summary-value";
        valueSpan.textContent = item.value;

        row.appendChild(labelSpan);
        row.appendChild(valueSpan);
        summaryBox.appendChild(row);
      });
    }

    // Show success feedback card
    if (successCard) {
      successCard.removeAttribute("hidden");
      successCard.classList.add("active");
      successCard.focus();
      successCard.scrollIntoView({ behavior: "smooth", block: "start" });
    }

    // Announce success to screen readers
    const alertLiveRegion = document.getElementById("formLiveAlert");
    if (alertLiveRegion) {
      alertLiveRegion.textContent = `Registration demonstration successful for
${sanitizedName}. All frontend validation checks passed!`;
    }

    // NOTE: Password is NEVER logged or displayed in summary.
    console.log(`[Form Validated Successfully] Profile created for
${sanitizedName} (${sanitizedId})`);
  });
}

//
================================================================================
// 5. INITIALIZATION & EXPORTS
//
================================================================================

// Run in browser when DOM is ready
if (typeof document !== "undefined") {
  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", initRegistrationForm);
  } else {
    initRegistrationForm();
  }
}

// Export functions for automated unit testing in Node.js
if (typeof module !== "undefined" && module.exports) {
  module.exports = {
    VALIDATION_CONFIG,
```

```
    validateFullName,
    validateStudentId,
    validateEmail,
    validateBranch,
    validateYear,
    validatePassword,
    validateConfirmPassword,
    validateTerms
  };
}
```